

TECHNICAL UNIVERSITY OF MOLDOVA

Faculty of Computers, Informatics and Microelectronics

Software Engineering

OPERATING SYSTEMS

Laboratory Work No. 10

Add Your Own System Call: trace

Performed by:
Bianca-Andreea Tataroi
Group FAF-242

Checked by:
Patricia Reițman

Chisinau, 2026

Contents

1 Introduction **2**

2 Objectives **2**

3 Part 1 - The Number and User Side **2**
3.1 Adding the System Call Number 2
3.2 Declaring the System Call 3
3.3 Adding the Assembly Stub 3
3.4 Creating the User Program 3
3.5 Modifying the Makefile 3
3.6 Testing Part 1 4

4 Part 2 - Kernel Implementation **4**
4.1 Adding the Tracing Mask 4
4.2 Implementing the Kernel Handler 5
4.3 Registering the System Call 5
4.4 Adding System Call Names 5
4.5 Printing Traced System Calls 5
4.6 Inheritance of the Tracing Mask 6
4.7 Testing Part 2 6

5 Part 3 - Testing Different Masks **7**
5.1 Understanding the Tracing Mask 7
5.2 Tracing All System Calls 7
5.3 Tracing Only fork() 8
5.4 Tracing Only write() 11
5.5 Tracing open() and close() 11

6 Answers to Observation Questions **12**
6.1 Question 1 12
6.2 Question 2 12
6.3 Question 3 12
6.4 Question 4 13

7 Conclusion **13**

8 GitHub Repository **13**

1 Introduction

System calls provide an interface between user-space programs and the operating system kernel.

They allow programs to request services such as file operations, process management, and memory allocation.

In the previous laboratory work, the Linux utility `strace` was used to observe system calls executed by programs.

In this laboratory, a similar tracing mechanism was implemented directly inside the xv6 operating system.

The purpose was to introduce a new system call named `trace()`, allowing processes to monitor selected system calls using a tracing mask.

The implementation required modifications to both the user-space interface and the operating system kernel.

2 Objectives

The objectives of this laboratory work were:

- Understand how system calls are registered and executed in xv6.
- Introduce a new system call named `trace()`.
- Assign the new system call number 23.
- Modify the user-space interface.
- Implement a kernel handler for tracing.
- Use bitmasks to select system calls.
- Allow child processes to inherit the tracing configuration.
- Test tracing using different commands.

3 Part 1 - The Number and User Side

3.1 Adding the System Call Number

The first step was to register the new system call in the xv6 operating system.

The file `kernel/syscall.h` was modified by adding:

```
1 #define SYS_trace 23
```

Each system call has a unique identifier used by the kernel to determine which operation must be performed.

In this xv6 version, number 22 was already assigned to `sync()`, so number 23 was assigned to `trace()`.

3.2 Declaring the System Call

The file `user/user.h` was modified by adding:

```
int trace(int);
```

This declaration allows user programs to invoke the new system call.

3.3 Adding the Assembly Stub

The file `user/usys.pl` was modified by adding:

```
entry("trace");
```

This entry allows the build system to generate the assembly instructions needed to invoke the system call.

The generated stub places the system call number in register `a7` and executes the `ecall` instruction.

3.4 Creating the User Program

A new file named `user/trace.c` was created.

The program receives a tracing mask and a command as arguments.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2,
      "usage: trace mask command [args...]\n");
    exit(1);
  }

  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }

  exec(argv[2], argv + 2);

  fprintf(2,
    "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

The program first activates tracing using the specified mask.

The `exec()` function then replaces the current program with the requested command.

The tracing configuration remains associated with the process.

3.5 Modifying the Makefile

The following entry was added to the `UPROGS` section:

```
$U/_trace\
```

This includes the new program in the xv6 file system.

3.6 Testing Part 1

The operating system was compiled and launched using:

```
make qemu
```

Inside the xv6 shell, the following command was executed:

```
trace 32 grep hello README
```

The initial output was:

```
3 trace: unknown sys call 23
trace: failed
```

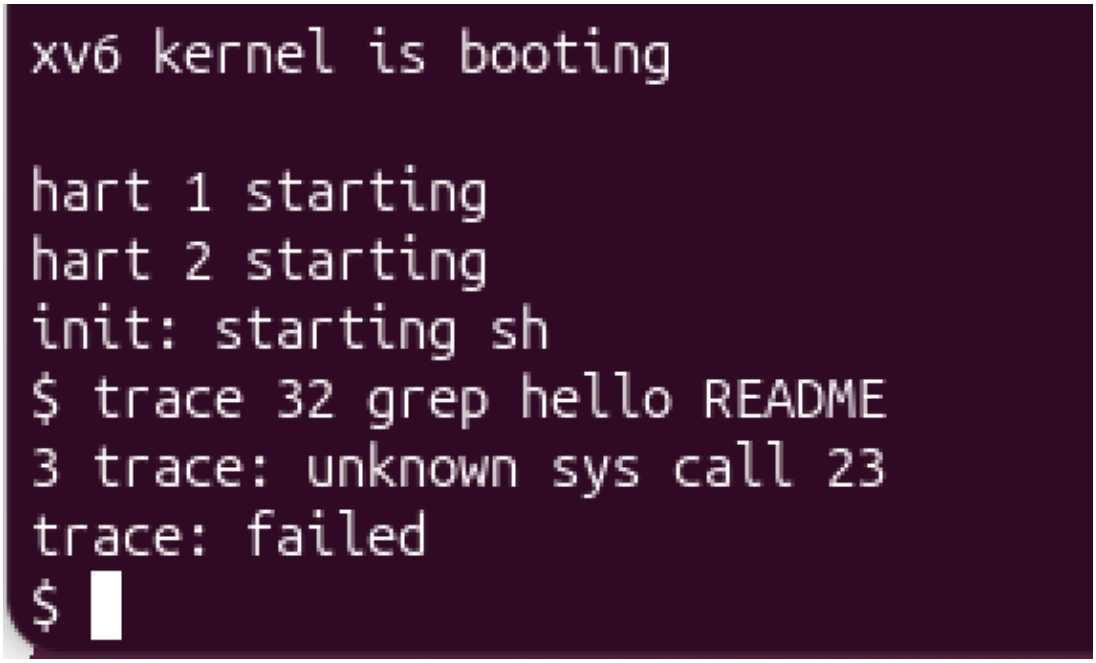


Figure 1: Initial test showing unknown system call 23

This result was expected because the kernel handler had not yet been implemented.

The message confirmed that the user program successfully sent system call number 23 to the kernel.

4 Part 2 - Kernel Implementation

4.1 Adding the Tracing Mask

The file `kernel/proc.h` was modified.

The following variable was added to the process structure:

```
1 int trace_mask;
```

Each process stores its tracing mask, which determines which system calls must be displayed.

4.2 Implementing the Kernel Handler

The file `kernel/sysproc.c` was modified by adding:

```
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

The `argint()` function retrieves the tracing mask from the system call arguments.
The `myproc()` function returns the currently running process.
The mask is stored in the process structure.
The function returns 0 to indicate successful execution.

4.3 Registering the System Call

The file `kernel/syscall.c` was modified.
The new handler was declared:

```
extern uint64 sys_trace(void);
```

The following entry was added to the system call table:

```
[SYS_trace] = sys_trace,
```

This connects the system call number 23 to the kernel function `sys_trace()`.

4.4 Adding System Call Names

A table named `syscallnames` was introduced.
The table associates system call numbers with readable names.
Examples include:

```
static char *syscallnames[] = {
  [SYS_fork] "fork",
  [SYS_read] "read",
  [SYS_open] "open",
  [SYS_write] "write",
  [SYS_close] "close",
  [SYS_trace] "trace",
};
```

The complete implementation includes all supported system call names.

4.5 Printing Traced System Calls

The `syscall()` function was modified to check whether tracing is enabled.
The following code was added:

```
p->trapframe->a0 = syscalls[num]();

if ((p->trace_mask >> num) & 1)
  printk("%d: syscall %s -> %ld\n",
```

```
5           p->pid,
6           syscallnames[num],
7           p->trapframe->a0);
```

The expression

```
(p->trace_mask >> num) & 1
```

checks whether the bit associated with the current system call is enabled.
If it is enabled, the kernel prints the process ID, system call name, and return value.
The function `printk()` is used for kernel output.

4.6 Inheritance of the Tracing Mask

The file `kernel/proc.c` was modified inside the `fork()` function.
The following line was added:

```
1 np->trace_mask = p->trace_mask;
```

This allows child processes to inherit the tracing mask from their parent process.
Therefore, tracing remains active when new processes are created.

4.7 Testing Part 2

After implementing the kernel functionality, xv6 was compiled and started again.
The command was executed:

```
trace 32 grep hello README
```

The following output was obtained:

```
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
```

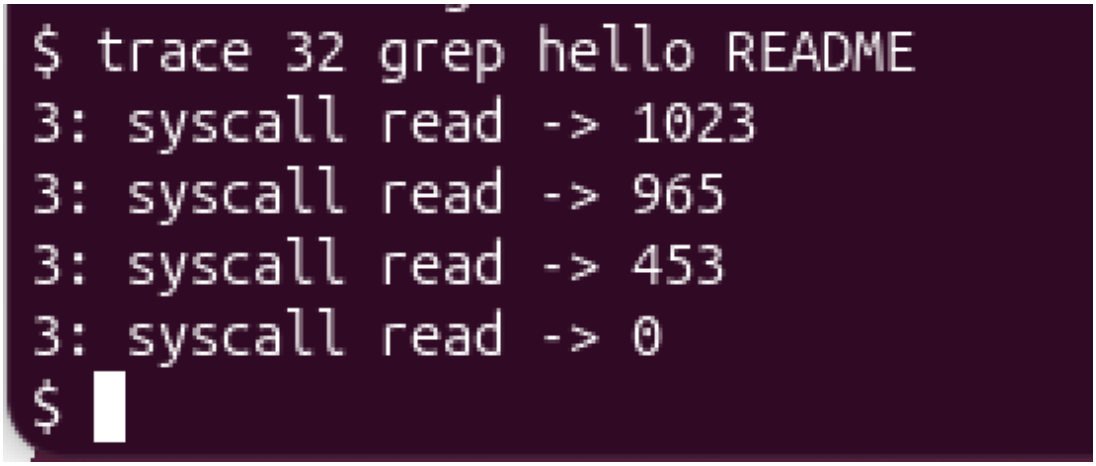


Figure 2: Successful tracing of read system calls

The tracing mechanism successfully displayed the `read()` system calls.
The returned values indicate the number of bytes read.

$$1023 + 965 + 453 = 2441$$

The final `read()` returned 0, indicating the end of the file.
This result matches the expected size of the README file.

5 Part 3 - Testing Different Masks

5.1 Understanding the Tracing Mask

A tracing mask determines which system calls are displayed.
Each system call corresponds to a bit in the mask.
For a single system call, the mask is calculated using:

$$\text{Mask} = 2^n$$

where n is the system call number.

System Call	Number	Mask
fork	1	2
read	5	32
open	15	32768
write	16	65536
close	21	2097152

Table 1: System call numbers and tracing masks

5.2 Tracing All System Calls

The following command was executed:

```
trace 2147483647 grep hello README
```

The mask enables bits 0 through 30.
Therefore, all implemented system calls with numbers in this range can be traced.



Figure 3: Tracing all system calls

The output showed the following system calls:

- trace
- exec
- open
- read
- close

The result demonstrates that the kernel correctly identifies and displays multiple types of system calls.

5.3 Tracing Only fork()

The following command was executed:

```
trace 2 usertests forkforkfork
```

The mask 2 corresponds to:

$$2^1 = 2$$

Therefore, only the `fork()` system call was traced.
The test creates several processes until the process table becomes full.

```
$ trace 2 usertests forkforkfork
usertests starting
test forkforkfork: 5: syscall fork -> 6
6: syscall fork -> 7
7: syscall fork -> 8
7: syscall fork -> 9
7: syscall fork -> 10
7: syscall fork -> 11
7: syscall fork -> 12
8: syscall fork -> 13
8: syscall fork -> 14
7: syscall fork -> 15
8: syscall fork -> 16
8: syscall fork -> 17
7: syscall fork -> 18
10: syscall fork -> 19
10: syscall fork -> 20
7: syscall fork -> 21
7: syscall fork -> 22
7: syscall fork -> 23
8: syscall fork -> 24
8: syscall fork -> 25
7: syscall fork -> 26
8: syscall fork -> 27
8: syscall fork -> 28
7: syscall fork -> 29
8: syscall fork -> 30
7: syscall fork -> 31
8: syscall fork -> 32
7: syscall fork -> 33
8: syscall fork -> 34
7: syscall fork -> 35
8: syscall fork -> 36
8: syscall fork -> 37
8: syscall fork -> 38
7: syscall fork -> 39
8: syscall fork -> 40
```

Figure 4: Tracing fork system calls during process creation

The output shows different process identifiers.
This confirms that child processes inherit the tracing mask from their parent process.

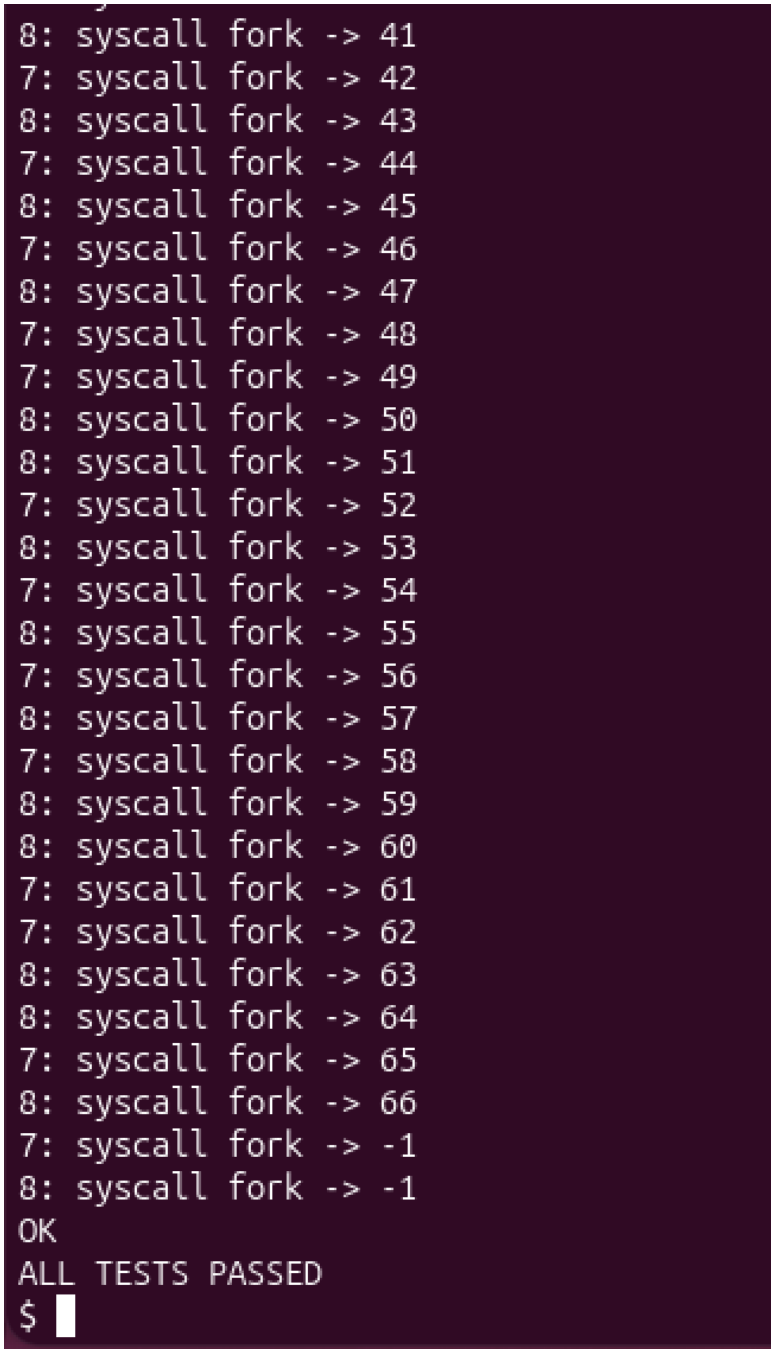


Figure 5: Fork tracing and successful completion of the test

At the end of the test, some `fork()` calls returned -1.

This indicates that the system could not create additional processes because the process limit was reached.

The test ended with:

```
OK
ALL TESTS PASSED
```

This confirms that the test completed successfully.

5.4 Tracing Only write()

The following command was executed:

```
trace 65536 echo hi
```

The mask corresponds to:

$$2^{16} = 65536$$

Therefore, only `write()` calls were traced.

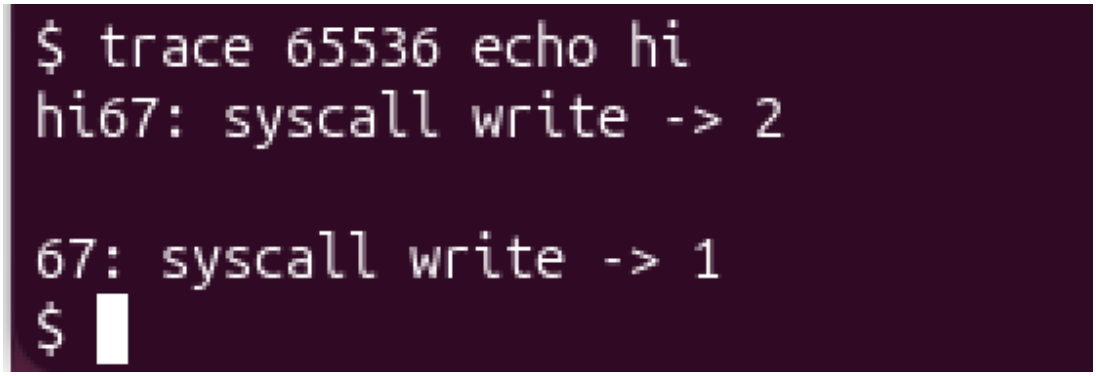


Figure 6: Tracing write system calls

The output showed two write operations:

```
67: syscall write -> 2
67: syscall write -> 1
```

The first call wrote the characters `hi`, containing two bytes.

The second call wrote the newline character, containing one byte.

The tracing output appeared between the text and its newline because the kernel printed a message after each system call.

5.5 Tracing open() and close()

The last test used a mask that enabled both `open()` and `close()`.

Their system call numbers are:

$$SYS_open = 15$$

$$SYS_close = 21$$

The combined mask is:

$$2^{15} + 2^{21}$$

$$32768 + 2097152 = 2129920$$

The following command was executed:

```
trace 2129920 grep hello README
```

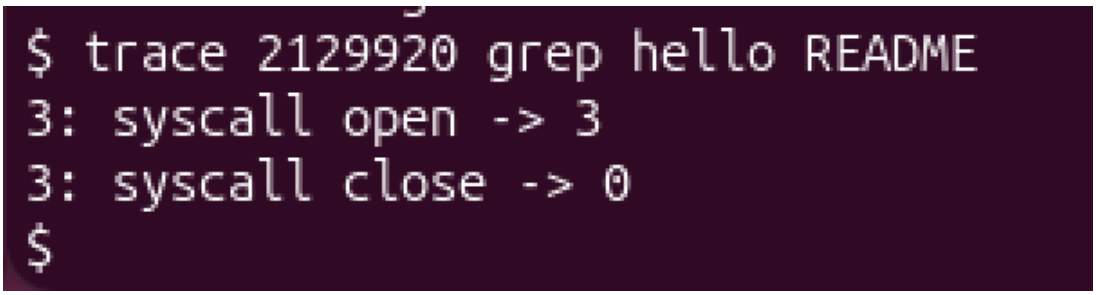


Figure 7: Tracing only open and close system calls

The output was:

```
3: syscall open -> 3
3: syscall close -> 0
```

This confirms that only the selected system calls were displayed.

6 Answers to Observation Questions

6.1 Question 1

Why does trace 32 grep hello README show only read, and not open or close?

The mask 32 corresponds to:

$$32 = 2^5$$

System call number 5 is `read()`.
Therefore, only the bit associated with `read()` is enabled.
The bits associated with `open()` and `close()` are not enabled.

6.2 Question 2

Why do several PIDs appear in the forkforkfork test?

Child processes inherit the tracing configuration from their parent process.
This functionality was implemented using the following line:

```
1 np->trace_mask = p->trace_mask;
```

As a result, all child processes can produce tracing output.

6.3 Question 3

Why is the tracing message printed between hi and the newline?

The `echo` program performs two `write()` calls.
The first call writes two bytes, representing the characters `hi`.
After the first call completes, the kernel prints its tracing message.
The second call writes one byte, representing the newline character.
Therefore, the tracing output appears between the text and the newline.

6.4 Question 4

Which mask traces both open and close, and nothing else?

The system call numbers are 15 for `open()` and 21 for `close()`.

The mask is:

$$2^{15} + 2^{21}$$

$$32768 + 2097152 = 2129920$$

Therefore, the required mask is:

$$\boxed{2129920}$$

7 Conclusion

During this laboratory work, a new system call named `trace()` was successfully implemented in the xv6 operating system.

In Part 1, the new system call was assigned number 23 and registered on the user side.

The initial test confirmed that the kernel received the new system call number.

In Part 2, the kernel implementation was completed by adding a tracing mask, system call handler, system call names, and tracing output.

The successful execution of

```
trace 32 grep hello README
```

confirmed that the implementation correctly traced read operations.

The returned values were 1023, 965, 453, and 0.

In Part 3, different tracing masks were tested.

The tests demonstrated tracing of read, write, open, close, and fork operations.

The forkforkfork test completed successfully with the message `ALL TESTS PASSED`.

This laboratory helped demonstrate how system calls are registered, executed, and monitored inside an operating system kernel.

It also illustrated how process attributes can be inherited by child processes.

8 GitHub Repository

The modified xv6 operating system source code, trace implementation, and laboratory notes were uploaded to GitHub.

The repository is available at:

https://github.com/Biancatataroi/Operating-Systems-Lab10